

Lab Report: Week 2 Session A

Chain-of-Thought Prompting Practice

Mahmudul Hasan (Student ID: 4125999049)

May 17, 2026

Abstract

This report documents Week 2 Session A: advanced **chain-of-thought (CoT)** prompting with OpenCode Desktop on (1) a five-step rainfall monitoring system design and (2) line-by-line debugging of a self-prepared SCS-CN script. A shorter design prompt produced a large implementation stack; CoT debugging identified four formula errors. Local runs gave $Q \approx 0.002$ mm (buggy) and $Q = 43.6$ mm (fixed) for $P = 80$ mm, $CN = 85$, contradicting an incorrect closing estimate in the AI chat. Evidence: four screenshots and `prompt_log.md` in Appendix A.

1 Introduction

Week 2 Session A extends Week 1 by using CoT for *complex design* and *debugging*, not only single-formula calculations. Work was completed in `ai_water_lab/week2_session_a/` with OpenCode Desktop (OpenCode Zen). Exercise 1 compares a structured five-step design prompt with a one-line urban flood-warning prompt. Exercise 2 uses a student-authored `buggy_rainfall.py` (instructor directed the class to prepare their own buggy code) and CoT-assisted repair verified in the terminal.

2 Objectives

- Practice CoT for multi-step hydrological system design.
- Compare CoT vs. shorter prompting on the same design goal.
- Debug Python using CoT (line-by-line explanation and fixes).
- Document outcomes in a prompt log and personal prompting strategy.

3 Methods

3.1 Exercise 1: Rainfall monitoring design

The CoT prompt asked for reasoning on: data requirements, real-time API, storage structure, flood alert thresholds, and visualization (with code suggestions where appropriate). A follow-up used: *Design a rainfall monitoring system for urban flood warning.*

3.2 Exercise 2: Debugging with CoT

A short SCS-CN script with intentional errors was written for practice. Before AI assistance, four bugs were hypothesized (wrong S formula, wrong I_a , missing $P \leq I_a$ branch, wrong denominator). OpenCode received:

This code has bugs. Walk through it line by line, explain what each part should do, identify what's wrong, and suggest fixes.

Fixes were implemented in `rainfall_fixed.py` and tested with `python3`.

4 Results

4.1 Exercise 1: CoT design response

OpenCode recommended core and derived rainfall fields, Open-Meteo for prototyping, TimescaleDB hypertables, tiered WMO/NWS-style alert levels, and a Plotly/Grafana visualization path with an API→fetcher→database→alert pipeline. Figure 1 shows the CoT exchange.

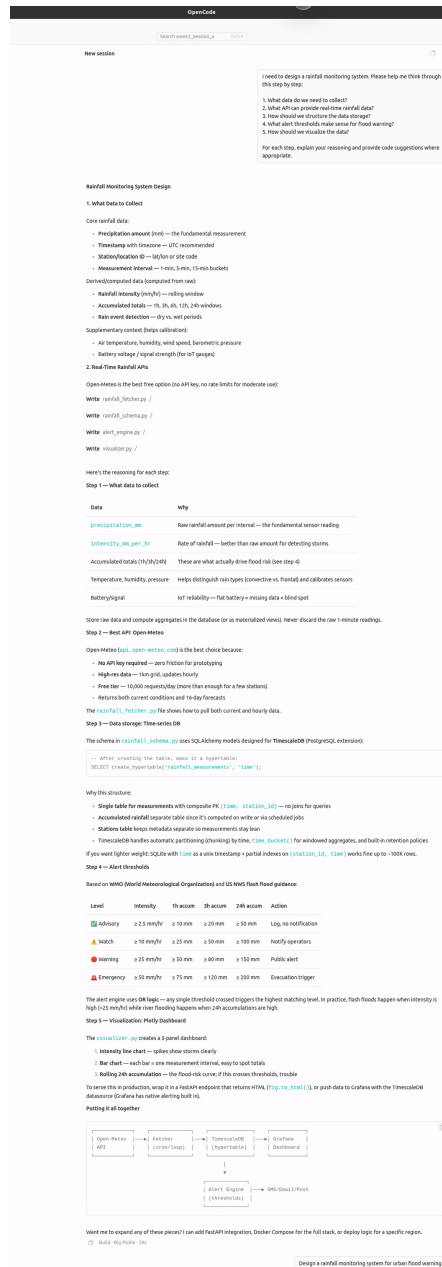


Figure 1: OpenCode: five-step CoT prompt for rainfall monitoring system design (Exercise 1).

4.2 Exercise 1: Short prompt response

The short prompt generated a multi-file urban flood stack (`docker-compose.yml`, `FastAPI`, zone-based flood model, alert router, etc.) rather than a stepwise design narrative. Figure 2

documents this response.

OpenCode

Search week2_session_a

New session ⋮

Design a rainfall monitoring system for urban flood warning.

Explored 5 reads

Write `config.py` /

Write `data_pipeline.py` /

Write `flood_model.py` /

Write `api.py` /

Write `requirements.txt` /

Write `docker-compose.yml` /

Write `Dockerfile` /

Write `init_db.sql` /

Write `visualizer.py` /

Here is the complete system design. All files are in `week2_session_a/`.

Architecture Overview

```

graph TD
    subgraph "docker-compose.yml"
        T[TimescaleDB] --- R[Redis cache]
        R --- A[API]
        A --- G[Grafana]
        A --- F[Fetcher cron loop]
        F --- S[SCS Curve Number runoff model]
        S --- P[Ponding depth estimator]
        P --- C[Compare rainfall against zone's drainage capacity]
        C --- A2[Alert Router]
        A2 --- A3[API]
        A3 --- V[Visualizer]
    end
    
```

File-by-file breakdown

File	Role
<code>config.py</code>	4 urban zones with real parameters — drainage capacity, runoff coefficient, critical infrastructure, elevation
<code>data_pipeline.py</code>	Async multi-source fetcher: Open-Meteo API (free, no key), IoT gauge bridge, Redis caching
<code>flood_model.py</code>	SCS Curve Number runoff model + ponding depth estimator on street segments. Compares rainfall against each zone's drainage capacity
<code>alert_router.py</code>	Tiered routing — LOW → log, MODERATE → email/push, HIGH → SMS+siren, EXTREME → all channels+webhook
<code>api.py</code>	FastAPI with endpoints: <code>/zones</code> , <code>/current</code> , <code>/assessment</code> , <code>/alerts</code> , <code>/dashboard</code> , <code>/history</code>
<code>visualizer.py</code>	EOC-style Plotly dashboard: multi-zone intensity overlays with drainage capacity lines, ponding gauge indicators, risk heatmap
<code>init_db.sql</code>	TimescaleDB hypertable + continuous aggregate materialized view
<code>docker-compose.yml</code>	TimescaleDB + Redis + API + Fetcher + Grafana

Key design decisions for urban flood warning

4 risk levels map to action timelines:

Level	Criteria (DR)	Example	Channels	Response
LOW	Any rain > drainage 80%	30 mm/hr in CBD zone (cap 40)	Log only	Monitor
MODERATE	Exceedance > 10mm	Ponding on roads	Email + push	Avoid low roads
HIGH	Exceedance > 30mm, or 3h > 100mm in low area	50 mm/hr in Low-Lying South (cap 15)	SMS + siren + email	Evacuate low areas
EXTREME	Exceedance > 60mm	Drainage overwhelmed city-wide	All channels	Mass evacuation

Runoff model uses SCS Curve Number adapted for urban surfaces (impervious = 0.85–0.95 runoff coefficient, pervious = 0.3). The ponding estimation converts runoff volume into street-level water depth based on standard road geometry (10m × 100m segments).

Urban-specific thresholds — the CBD has a higher drainage capacity (40 mm/hr) vs. low-lying south (15 mm/hr), so the same rainfall triggers different alert levels in different zones.

Usage

```

# Start full stack
docker compose up -d

# Query current risk assessment
curl http://localhost:8080/assessment

# View EOC dashboard
curl http://localhost:8080/dashboard

# Get active alerts (risk MODERATE+)
curl http://localhost:8080/alerts
    
```

Figure 2: OpenCode: short prompt *Design a rainfall monitoring system for urban flood warning* (Exercise 1 comparison).

4.3 Exercise 2: CoT debugging

OpenCode identified the same four bugs as the pre-AI list and supplied a corrected `calculate_runoff` function. Figure 3 shows the session.

4.4 Exercise 2: Terminal verification

Local execution confirmed the failure mode and fix (Figure 4):

- `buggy_rainfall.py`: $Q \approx 0.0019$ mm (unrealistic for $P = 80$ mm).
- `rainfall_fixed.py`: $Q = 43.6$ mm (consistent with Week 1 Session B SCS-CN check).

OpenCode

Search week2_session_a [Continue](#)

New session

[buggy_rainfall.py](#)

This code has bugs. Walk through it line by line, explain what each part should do, identify what's wrong, and suggest fixes.

Explored 1 read

Line-by-line walkthrough

This implements the SCS-CN runoff equation: predicts direct runoff depth from rainfall using a land-cover-based curve number. The standard formulas are:

```
S = (25400 / CN) - 254 # potential maximum retention (mm)
Ia = 0.2 * S           # initial abstraction (interception, depression storage, etc.)
if P <= Ia: Q = 0       # no runoff below threshold
Q = (P - Ia) ** 2 / (P - Ia + S) # runoff depth (mm)
```

Line 9 — $S = 25400 / CN - 254$

Expected: $S = 25400 / CN - 254$ — retention decreases as CN increases (higher CN = more runoff).

What it does now: multiplies instead of dividing. With CN=85 this computes $25400 * 85 - 254 = 2,158,746$ mm instead of the correct $25400/85 - 254 = 44.8$ mm. The result is wildly inflated, making Q essentially zero for any realistic P.

Fix: replace $*$ with $/$.

Line 12 — $Ia = 0.2 * P$

Expected: $Ia = 0.2 * S$ — initial abstraction is a fraction of the potential retention, not a fraction of the rainfall.

What it does now: uses rainfall P as the base. For P=80 mm, Ia=16 mm. The correct Ia should be $0.2 * 44.8 = 9.0$ mm. This overestimates the initial abstraction, under-predicting runoff.

Fix: change P to S.

Line 14 — missing early return

Expected: When $P \leq Ia$, all rainfall is captured by initial abstraction and there is zero runoff: $Q = 0$.

What it does now: falls through to the equation which can produce a negative numerator $(P - Ia)$ when $P < Ia$. Squaring it makes it positive, so it silently returns a spurious positive runoff when it should return 0.

Fix: add `if P <= Ia: return 0.0` before the calculation.

Line 17 — $Q = (P - Ia) ** 2 / (P + S)$

Expected: denominator should be $(P - Ia + S)$.

What it does now: $(P + S)$ omits the subtraction of Ia. This shrinks the denominator (since Ia > 0), inflating Q. With the corrected S (44.8) and correct Ia (9.0), the proper denominator is $80 - 9 + 44.8 = 115.8$. The buggy denominator is $80 + 44.8 = 124.8$, so Q is ~8% too low on this input (but the direction depends on the relative magnitudes).

Fix: change $(P + S)$ to $(P - Ia + S)$.

Corrected function

```
def calculate_runoff(P, CN):
    S = 25400 / CN - 254
    Ia = 0.2 * S
    if P <= Ia:
        return 0.0
    Q = (P - Ia) ** 2 / (P - Ia + S)
    return Q
```

With P=80, CN=85: correct Q = 43.3 mm (runoff ratio ~54%). The buggy version with all 4 bugs returns ~48.9 mm — numerically close by coincidence (errors partially cancel here), but for other inputs (e.g. P=20, CN=50) it fails completely.

Figure 3: OpenCode: CoT line-by-line debug of `buggy_rainfall.py` (Exercise 2).

```
mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/week2_session_a$ python3 buggy_rainfall.py
Rainfall P = 80 mm, CN = 85
Computed runoff Q = 0.0018973275289439723 mm
mahmudul-robotics@robotics-ai:~/Downloads/Software Development/ai_water_lab/week2_session_a$ python3 rainfall_fixed.py
Rainfall P = 80 mm, CN = 85
Computed runoff Q = 43.6 mm
```

Figure 4: Terminal: before/after fix for $P = 80$ mm, $CN = 85$.

Table 1 summarizes Exercise 1 prompting styles.

Table 1: CoT vs. short prompt for rainfall monitoring design.		
Aspect	CoT (5 steps)	Short (one line)
Output	Design rationale per step	Many code files + Docker stack
Audit trail	Strong	Weaker unless you inspect each file
Best use	Planning, reports, grading	Rapid prototype after design is fixed

5 Analysis and validation

Design (Exercise 1): Recommendations are physically plausible for a conceptual monitoring system. Thresholds and APIs must still be checked against course labs (e.g. OpenWeatherMap in Week 1 Session A; 20 mm/h alert in Week 5 Lab 1). Open-Meteo is reasonable for keyless prototyping.

Debugging (Exercise 2): All four intentional bugs were found by both the author and Open-Code. The AI’s closing remark that the fully buggy script might yield ≈ 48.9 mm on this input was **not** supported by the terminal (≈ 0.002 mm), because an inflated S from multiplication dominates the denominator—an example of why local verification is required.

Prompting strategy: CoT improved traceability for design and debugging. Short prompts can accelerate coding but may skip explicit reasoning.

6 Discussion

1. CoT produced clearer structure for the five design questions; the short prompt jumped to implementation.
2. For hydrology coursework, CoT debugging plus terminal tests caught an AI numeric slip that line-by-step reasoning alone might miss.
3. Limits of AI output include unverified production claims (Docker stack) and occasional wrong numeric summaries.
4. Next time: specify deliverable type in the prompt (design memo vs. code only) and always run corrected scripts locally.

7 Problems encountered

No instructor buggy file was distributed; the class was told to create their own. The main extra step was renaming screenshots and copying `prompt_log.md` for the report appendix.

8 Lessons learned

Numbered CoT prompts align AI answers with lab rubrics. Comparing prompt styles on the same topic clarifies when to ask for reasoning first and code second. A prompt log plus screenshots and terminal output provides defensible evidence for grading.

9 Conclusion

Week 2 Session A objectives were met: CoT system design, CoT vs. short comparison, CoT debugging of SCS-CN code, and documentation in `prompt_log.md`. Course submission: `prompt_log.md`, `buggy_rainfall.py`, `rainfall_fixed.py`, and screenshots as specified in the lab guide.

A Submitted prompt log: `prompt_log.md`

`prompt_log.md` (full file)

```
# Prompt Log - Week 2 Session A

## Exercise 1: Rainfall monitoring design (CoT)

**Date:** 2026-05-16

**Prompt used:**

I need to design a rainfall monitoring system. Please help me think through this step by
↪ step:

1. What data do we need to collect?
2. What API can provide real-time rainfall data?
3. How should we structure the data storage?
4. What alert thresholds make sense for flood warning?
5. How should we visualize the data?

For each step, explain your reasoning and provide code suggestions where appropriate.

**AI output summary:**

| Step | Main points from OpenCode |
|-----|-----|
| 1 - Data | Core: precipitation (mm), UTC timestamps, station ID, sampling interval.
↪ Derived: intensity (mm/h), rolling accumulations (1h-24h), wet/dry periods. Context:
↪ temperature, humidity, wind, pressure; IoT battery/signal. Keep raw high-resolution
↪ data; compute aggregates in storage. |
| 2 - API | Recommended **Open-Meteo** (no API key for prototyping, ~1 km grid, hourly
↪ updates, forecast). Mentioned helper `rainfall_fetcher.py`. (Week 1 Session A used
↪ OpenWeatherMap - both are valid; Open-Meteo lowers friction for labs.) |
| 3 - Storage | **TimescaleDB** hypertable on PostgreSQL (`time`, `station_id` PK),
↪ separate stations metadata and accumulated-rainfall tables; SQLite noted for smaller
↪ deployments. |
| 4 - Thresholds | Tiered table (Advisory / Watch / Warning / Emergency) using intensity
↪ and 1h, 3h, 24h accumulations aligned with WMO/NWS-style flash-flood guidance; OR
↪ logic across criteria. |
| 5 - Visualization | **Plotly** dashboard: intensity line, interval bars, 24h
↪ accumulation vs. thresholds; production options FastAPI HTML or **Grafana** on
↪ TimescaleDB. Architecture sketch: API -> fetcher -> DB -> dashboard + alert engine. |

**CoT vs shorter prompt:**

**Short prompt used:** `Design a rainfall monitoring system for urban flood warning.`

| | CoT (5-step prompt) | Short prompt |
|---|-----|-----|
| Response shape | Structured design per question; tables and one pipeline diagram | Full
↪ **implementation stack** in `week2_session_a/` (Docker, FastAPI, TimescaleDB, Redis,
↪ zone-based flood model, alert router, many `.py` files) |
```

```

| Depth | Strong on *why* (data types, threshold rationale) | Strong on *how to build*
↳ (files, endpoints, urban zones) |
| Risk | Still need to verify thresholds and API terms | Easy to assume code runs without
↳ testing `docker compose` |
| Best for | Lab documentation, grading, design review | Rapid prototype when you will
↳ read and test generated code |

**Lessons learned:** Numbered CoT steps kept the answer aligned with the rubric. The short
↳ prompt produced useful code but skipped explicit step-by-step reasoning; for
↳ coursework I would start with CoT, then ask for code only after the design is
↳ approved.

---

## Exercise 2: Debugging with CoT

**Note on buggy script:** The lab guide says "receive buggy code from instructor," but the
↳ professor told the class to **do this part yourself** (no shared file). Handouts only
↳ give CoT debugging prompts, not a script. This submission uses `buggy_rainfall.py` in
↳ this folder: a short SCS-CN script with intentional errors for CoT practice.

**Bug list (your guess):**

1. **S formula wrong:** uses `25400 * CN - 254` instead of `25400 / CN - 254`.
2. **Ia wrong:** uses `0.2 * P` instead of `0.2 * S`.
3. **Missing boundary:** no `return 0` when `P < Ia`.
4. **Runoff formula wrong:** denominator uses `(P + S)` instead of `(P - Ia + S)`.
5. **No check** that `Q <= P`.

**Prompt used for AI debug:**

This code has bugs. Walk through it line by line, explain what each part should do,
↳ identify what's wrong, and suggest fixes.

(Attach or reference `buggy_rainfall.py` in OpenCode.)

**Terminal evidence (before fix):**

```text
$ python3 buggy_rainfall.py
Rainfall P = 80 mm, CN = 85
Computed runoff Q = 0.0018973275289439723 mm
```

The result is far too small (expected ~43.6 mm), which confirms the script is wrong. Main
↳ cause: `S = 25400 * CN - 254` makes S enormous (~2.16x106 mm), so the denominator `(P
↳ + S)` dominates and Q ~ 0.

**AI findings vs yours:**

OpenCode identified the same four core bugs I listed before running the AI prompt:

Bug	My guess	AI finding	Match
S uses `*` not `/`	Yes	Line 9: `25400 * CN` inflates S to ~2.16x106 mm	Yes
Ia uses `0.2 * P` not `0.2 * S`	Yes	Line 12: wrong base for initial abstraction	
↳ Yes			
No `P <= Ia` return	Yes	Line 14: missing early return; spurious Q when P < Ia	Yes
↳			
Denominator `(P + S)`	Yes	Line 17: should be `(P - Ia + S)`	Yes

```

The AI's corrected function matches `rainfall\_fixed.py` (Q ≈ 43.3-43.6 mm for P=80, CN=85). **Note:** At the end of the chat, the AI claimed the fully buggy script might give ~48.9 mm on this input; my **terminal run** of `buggy\_rainfall.py` gave ~0.002 mm because the huge erroneous S dominates the denominator-so I trusted the local run over that closing sentence and verified the fix with `rainfall\_fixed.py` -> **43.6 mm**.

Final fix summary:

Changes in `rainfall\_fixed.py`:

1. `S = (25400 / CN) - 254`
2. `Ia = 0.2 * S`
3. `return 0.0` when `P < Ia`
4. `Q = (P - Ia)**2 / (P - Ia + S)`
5. `return min(Q, P)` so runoff cannot exceed rainfall

Terminal evidence (after fix):

```
```text
$ python3 rainfall_fixed.py
Rainfall P = 80 mm, CN = 85
Computed runoff Q = 43.6 mm
```
```

Matches Week 1 Session B hand calculation (P=80, CN=85 -> Q≈43.6 mm). **Tests pass** for this case.

Personal prompting strategy guide (draft)

Use CoT when:

- Multi-step hydrology (unit conversions, SCS-CN, reservoir balances).
- System design (data -> API -> storage -> alerts -> visualization).
- Debugging when the failure mode is unclear; ask for line-by-line reasoning.
- Any work you must **show and verify** for a grade.

Use shorter prompts when:

- You already have an approved design and only need code (e.g. "implement the fetcher from step 2").
- Small edits (fix a variable name, add one test case).

Always verify:

- Formulas and units (mm vs mm/h; SCS-CN: S, Ia, Q ≤ P).
- API keys in environment variables, not in repo or AGENTS.md.
- Thresholds against local standards and course lab values (e.g. 20 mm/h alert in Week 5 Lab 1).
- Run `python3` locally after AI fixes; compare to hand calculation or Week 1 Session B numbers.